

IGR Project

Sunset over an infinite sea

Alice JEANNIN

Introduction

For this project, I wanted to focus more on the aesthetic than trying to reproduce one or more specific methods I would have found in research papers. This project aims to render a sea under a sunset. The camera can be moved in such a way that the sea seems infinite. I used ray tracing for the rendering. My project focuses more on geometry and simulation (maybe more on geometry). The simulation consists in imitating waves and water behavior more in a descriptive way than in a physical way. For the geometry part, I created the mesh from scratch using code, and I update it each time the camera moves using an algorithm I found myself.

1. Ray tracing

1.1. Concrete implementation

I decided to do the computation of my ray tracer on the GPU. Basically, my vertex shader receives 2 triangles that represent the screen and passes to the fragment shader the direction of a ray to shoot for each fragment. The ray tracing is therefore performed in the fragment shader.

1.1.1. Storage buffer

To pass my vertices and triangles to the fragment shader, I used storage buffers. I wanted to interpret the buffer's data as vec3 array in the shader, but the problem was that in glsl, vec3 have to be aligned like vec4. If verticesPosition is a vec3 array, the address of verticesPositions[1] is the address of verticesPositions[0] + 4 * sizeof(float), while in C the address would be the address of verticesPositions[0] + 3 * sizeof(float). The solution I found was to add 1 float between each vec3 in the C struct before sending it to the shader.

1.1.2. Object Properties

To differentiate the objects, I simply passed another storage buffer called objectProperties which stored both the objects' properties (color, reflection/refraction ratio, diffuse/specular ratio) and the index of the last triangle. The challenge was to put all this information in a struct where the color would be well aligned (see Storage Buffer part above). Fortunately, I had 4 floats (4 bytes in glsl), 1 int (4 bytes in glsl) and 1 vec3 (3 * 4 bytes) which makes exactly 8 * 4 bytes (remember : vec3 has to be aligned on 4 * 4 bytes). To be sure the int will take 4 bytes in the C struct, I used the type int32_t from the stdint.h library.

Then each time I find the closest triangle intersected by a ray, I have to browse the list of objects to determine which object it is from, but it is not a big problem because I only have a couple of objects (only 1 being the sea in the final scene).

1.1.3. Recursion and trees

Unfortunately, you cannot have recursive functions in glsl. But ray tracing needs recursion to track all the rays obtained by refraction and reflection. I decided to implement a tree representing the cast rays. To do so I used an array of size $2 * (height + 1) - 1$. The nodes are stored in the array's cells. The sons' indices of the node of index i are $2 * i + 1$ and $2 * i + 2$. For each node I stored a struct containing information about the ray. I first computed the direction and origin of each reflected and refracted ray from root to leaves and then computed the final color from leaves to root.

1.2. Concrete computations

1.2.1. Computing the initial ray

We have one initial ray for each fragment of the quad I display. This ray comes from the position of the screen and points towards the world position of the corresponding point of the virtual screen in front of the camera. The origin of the ray is always the camera position and is passed to the fragment shader as a uniform value. I used the parameters of the camera (Fov, aspectRatio and up and right directions) to determine the direction of the ray of each fragment. Concretely, I computed the direction of the ray in the vertex shader for each corner of the quad with the following formula ((x, y) is the position of each vertex with x and y ranging from -1 to 1) :

$$\text{rayDirection} = \text{forward} + (x * \text{halfWidth}).\text{right} + (y * \text{halfHeight}).\text{up}$$

where

$$\text{halfHeight} = \tan\left(\frac{\text{Fov}}{2}\right)$$

$$\text{halfWidth} = \text{aspectRatio} * \text{halfHeight}$$

The direction is then interpolated for each fragment.

1.2.2. Computing other rays

To compute the reflected ray, I used the coordinates of the point of intersection and the normal n of the intersected triangle.

$$\text{reflectedRay} = 2 * (\text{dot}(-\text{ray_direction}, n)) * n + \text{ray_direction}$$

For the refracted ray, I change the origin of the ray and keep the same direction. I assume I have the same propagation medium everywhere, and thus the same refraction index. Concretely, I assume all my transparent surfaces are like very thin glass surfaces. In any case, I only used reflection for my final scene, which is enough since in the case of a real sea, the depth of the water is large enough for the observer to only see an opaque blue surface.

The last ray that is created is the shadow ray. It is shot toward the light source and is used to detect whether the point of intersection is in shadow.

1.2.3. Mixing color

To obtain the final color of a fragment, I mix the color obtained by the phong light model in the point of intersection and the colors computed recursively by shooting the reflected and refracted rays. As for the shadow ray, if it intersected a triangle, I divide the phong light model color by 2.

If no triangle is intersected, I simply return the color of the sky whose calculation is explained in the next paragraph.

1.2.4. Background

To find the color of the sky, I simply compute the dot product between the ray direction and the light direction and return the corresponding color in a color gradient ranging from yellow to blue.

```
float sun_closeness = dot(rayDirection, normalize(lightDirection));
if(sun_closeness>sunThreshold){
    return yellow;
}
else if(sun_closeness>haloThreshold){
    float x = (sun_closeness-haloThreshold)/(sunThreshold-haloThreshold);
    return getGradient(orange, yellow, x);
}
else if(sun_closeness>pinkSkyThreshold){
    float x = (sun_closeness - pinkSkyThreshold)/(haloThreshold - pinkSkyThreshold);
    return getGradient(pink, orange, x);
}
else{
    float x = (sun_closeness - -1)/(pinkSkyThreshold - -1);
    return getGradient(blue, pink, x);
}
```

1.3. Intermediate results

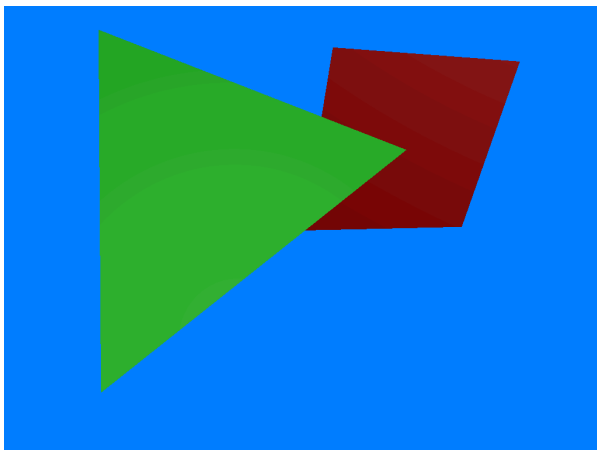


Figure 2: Light

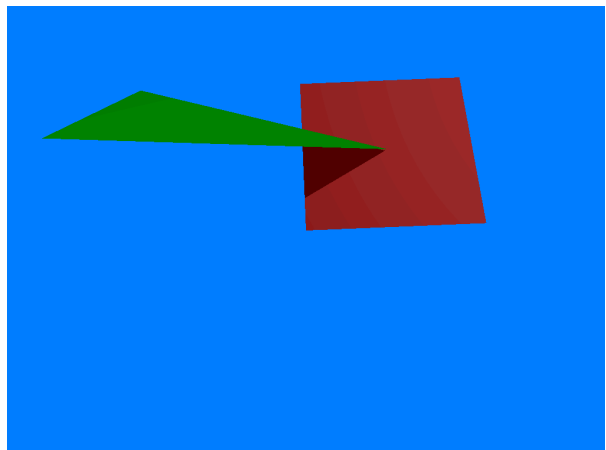


Figure 3: Shadow



Figure 5: reflection

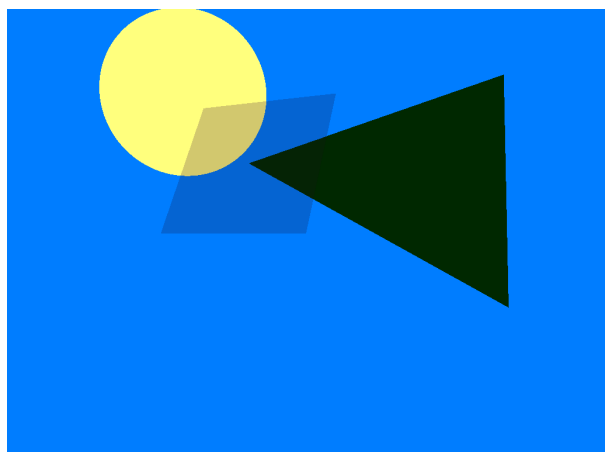


Figure 6: refraction

2. Sea geometry and simulation

2.1. Initial mesh

For the sea, I took inspiration from the video game Monument Valley. I split a rectangle into triangles (a regular grid, each cell being split into 2 triangles). The height of each vertex is random in a certain range, which gives us a nice blocky sea as you can see in the following figure.

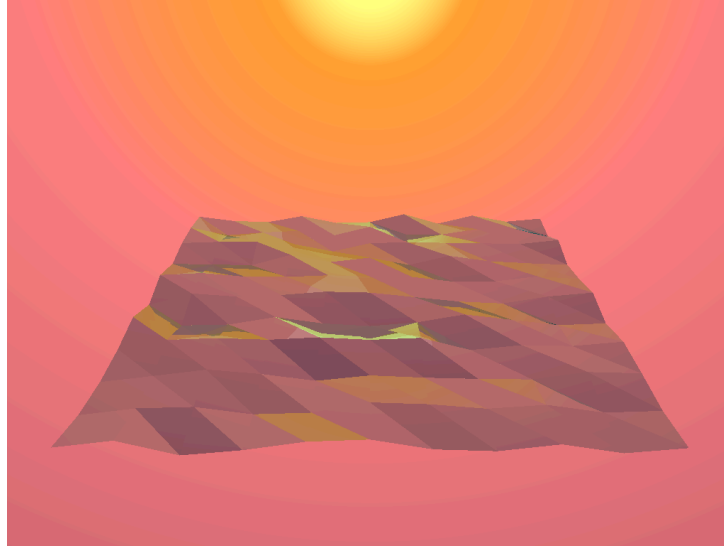


Figure 7: grid of vertices with random height



Figure 8: closer view on the sea

As you can see in the figure, the water looks pink while the mesh color is cyan. It is because of the reflection which represents 70% of the final mixed color.

2.2. Simulation

Then I attached a vertical spring to each vertex. I had to put a different spring constant to each vertex, otherwise, the vertices would be at height 0 at the same time, and then we would have an ugly flat sea for a moment and a brutal change of triangle colors when the triangles' orientations change at the same time.

$$\vec{F} = -k * y.\vec{u}_y$$

where k is the spring constant.

Unfortunately, with this animation, there is no coherence between the wave crests while in the sea you usually have huge waves in addition to small oscillation of the water. That's why I added a vertical offset to each vertex which is obtained with the plane wave formula:

$$\text{offset} = \cos(w * t - \vec{k}.\vec{x})$$

where w is the pulsation and $\vec{k}$ is the wave vector

In our simulation, $\vec{x}$ is the position in the horizontal (x, z) plane.



Figure 9: Waves after implementing the simulation

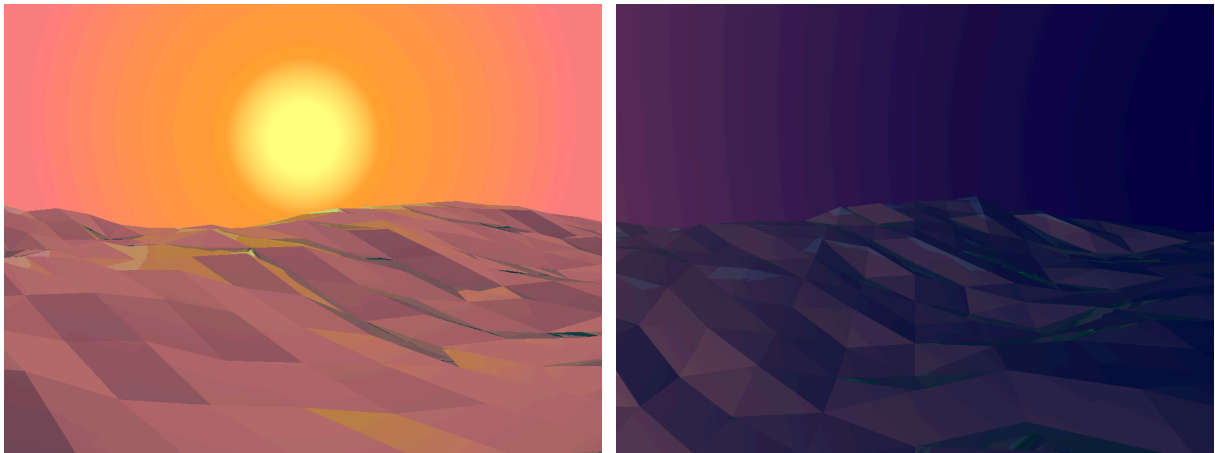


Figure 10: Waves with more triangles

3. Changing the mesh to simulate infinite surface

3.1. Conventions

For the next calculations in this document, I define the % operator for integers as described below:

$$a \% b = c \Leftrightarrow \begin{cases} a = k * b + c \\ k \in \mathbb{Z} \\ c \in [0, b[\end{cases}$$

Note that it is not the same definition than in C. In C, the result of % for a negative number will be a negative number, which is not the behavior expected for my formulas. That means I had to create a special function in my code to implement this operator.

3.2. Structures used for vertices and triangle indices

To understand the formulas to update the mesh, you should remember how my vertices and triangles are stored in the memory.

3.2.1. Vertices

My vertices and triangle indices are both stored in arrays. The vertices are stored in 1D array which should be interpreted as a 2D array. The indices (k, l) of each vertex represent its position in the initial grid. The actual index to access the vertex in the array computed this way:

$$\text{vertexIndex} = l + k * (N + 1)$$

where N is the number of columns in the grid.

You will notice that k and l range from 0 to N included and that the number of vertices is therefore 2^{N+1} .

3.2.2. Triangles

To render the sea, I need $2 * N^2$ triangles. However, as I will explain later in this document, the vertices are going to move from one side of the grid to the other, and the set of triangles that should exist or not will change over time. The way I will move the vertices (when I say “move”, I mean changing the (x, z) position, the vertex remains at the same index in the array), ensures that the only triangles I will have to render will be composed of neighboring vertices in the 2D array if we consider the (k, l) indices modulo N . You can think of my array as a grid in a snake game where you can go from the left wall to the right wall directly. Therefore, I decide to associate with each vertex of index n 2 triangles of indices $2 * n$ and $2 * n + 1$, whose vertex is the “left corner” of the square they form, even if the triangles should not be displayed at a time t .

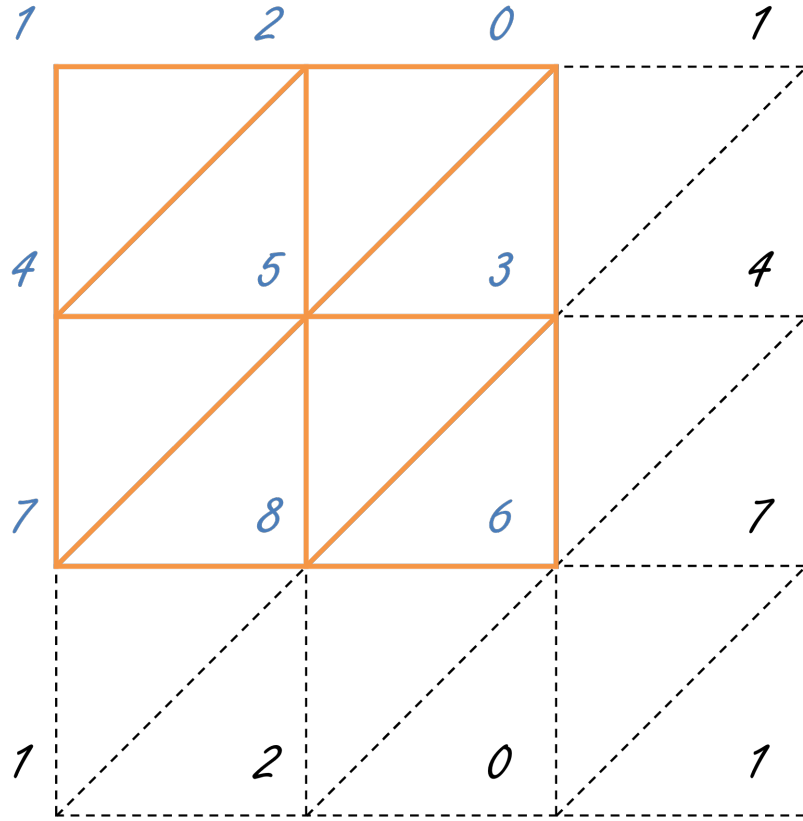


Figure 13: All possible triangles

If a triangle should not be displayed, I change its indices to $(0, 0, 0)$ and it is no longer visible. We can update the triangle associated with the vertex of indices (k, l) with the following code:

```

v1 = l + k*(N+1);
v2 = (l+1)%(N+1) + k*(N+1);
v3 = l + ((k+1)%(N+1))*(N+1);
v4 = (l+1)%(N+1) + ((k+1)%(N+1))*(N+1);
n = v1;
if(should_be_displayed[n]){
    triangles[2*n] = (v1, v2, v3);
    triangles[2*n+1] = (v2, v3, v4);
}
else{
    triangles[2*n] = (0, 0, 0);
    triangles[2*n+1] = (0, 0, 0);
}

```

I have $2 * 2^{N+1}$ triangles instead of 2^N but this structure is more convenient for what we want to do with it. If we are concerned about the ray tracing performance, we can easily copy the $2 * 2^N$ non-null values of this array into another one that we would pass to the ray tracer, but we would have to do so for each frame, or at least each time our mesh changes.

3.3. Grid position

What will happen concretely for our eyes when the camera moves is that the grid of vertices that represents the sea will also move after a certain threshold. The grid will always match the infinite grid obtained by extending the initial grid. We always move the vertices by step of $(N + 1) * S$ where S

is the side of a cell of the infinite grid, such as all points of the actual grid in front of the camera is occupied by exactly one vertex. The following figure illustrates this point.

Vertex index

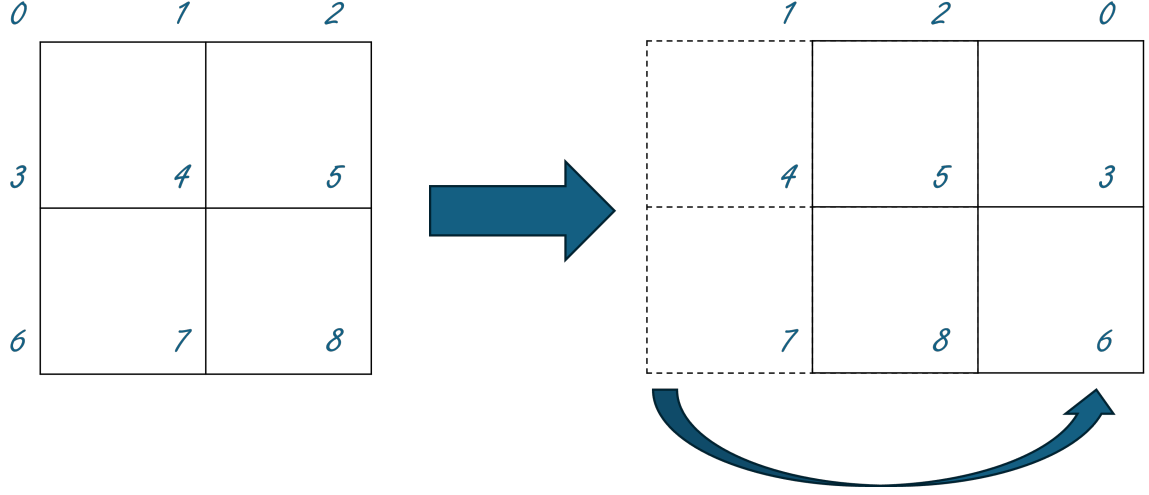


Figure 14: The grid of the sea moving

To know when and where to move our vertices, we compute the coordinates (i, j) of our vertices in this virtual grid that follows the camera. The formula is given below, x and z being the coordinates of the vertex in the (x, z) plane:

$$i = \text{round}\left(\frac{x - \text{camera}.x + \frac{W}{2}}{S}\right)$$

$$j = \text{round}\left(\frac{z - \text{camera}.z + \frac{W}{2}}{S}\right)$$

where S = the side of a cell of the grid

W = the width of the grid

If i or $j \notin [0, N]$, the vertex is currently outside the grid, and it has to be shifted in order to go back into it. The following formula can be used to compute its new coordinates:

$$x' = x + \left(\frac{i' - i}{N + 1}\right) \cdot S \cdot (N + 1)$$

$$z' = z + \left(\frac{j' - j}{N + 1}\right) \cdot S \cdot (N + 1)$$

$$\text{where } \begin{cases} i' = i \% (N + 1) \\ j' = j \% (N + 1) \end{cases}$$

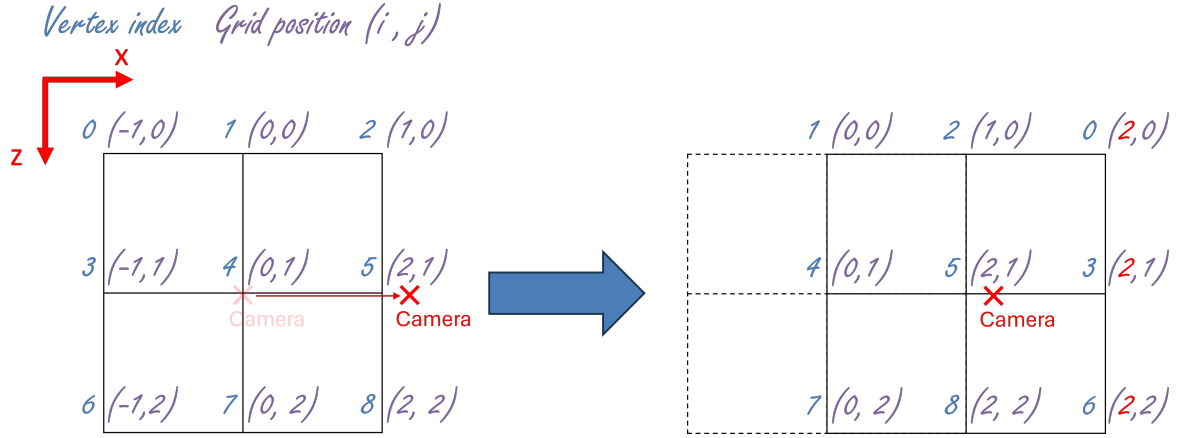


Figure 15: illustration of the grid positions

This grid position also tells us whether we should display the triangles associated with the vertex. In fact, if $i = N$ or $j = N$, we are on the border of the grid and the triangles should not be displayed, since the neighboring vertices are on the other side of the grid.

4. Performance and possible improvements

4.1. Ray tracing

For now, my ray tracing is rather slow. On my laptop, which doesn't have a good GPU, the simulation works in real time for my default scene which contains 242 triangles. But when I have about 1000 triangles, it becomes very slow. We should implement bounding boxes to reduce the number tested for each ray. To make it effective, we should have several boxes for our single mesh, but the problem is that our mesh is liable to change completely so we would have to recompute the bounding boxes frequently. It must be more efficient than it is now, nonetheless.

4.2. Animation computations

The way I update the mesh and implement my physics computations has the particularity of being completely parallelizable. The changes I make for each vertex and its associated triangles depend only on its own coordinates. For now, I use OpenMP for each loop to speed up the calculations, but I could implement them on the GPU, for example in a compute shader. In addition to accelerating the computations, we would not have to copy the vertex data from the CPU memory to the GPU one, which also takes a certain amount of time.